

Authored by : Arpit Raj , Lnmiit jaipur

Case Study 1 — College Management System

(UNF → 5NF)

Goal:

This is a complete, interview-level case study where we'll start from a poorly designed database and normalize it step by step until we reach the final schema. We'll explain every decomposition, every Functional Dependency (FD), and every decision made along the way.

Problem Statement

A college wants to maintain information about:

- Students
- Departments
- Faculty
- Courses
- Enrollments
- Classrooms
- Semesters

The administration initially stores everything in a single spreadsheet.

Identify Entities

Entity	Description
Student	Stores student information
Department	Stores department details
Faculty	Stores faculty information
Course	Stores course details
Enrollment	Student-Course relationship
Classroom	Room information
Semester	Semester details

Initial UNF (Unnormalized Form)

Suppose the college stores data like this:

StudentID	StudentName	Department	HOD	Phones	CourseIDs	CourseNames	Faculty
101	Aadya	ECE	Dr. Sharma	9876,9123	CS301,EC204	DBMS,Digital Electronics	Dr. Rao Gupta

Immediately notice:

- Multiple phones
- Multiple courses
- Multiple faculty names
- One giant table
- Huge redundancy

This is UNF.

Problems in UNF

- **Repeating Groups**
 - Phones

- CourseIDs
- CourseNames
- FacultyNames
- **Redundancy**
 - Department name repeated for every student.
 - Faculty repeated.
 - Semester repeated.
 - Room repeated.

Step 1 — Find Functional Dependencies

This is the most important step.

Student FDs

StudentID → StudentName
StudentID → DepartmentID

Department FDs

DepartmentID → DepartmentName
DepartmentID → HOD

Course FDs

CourseID → CourseName
CourseID → Credits
CourseID → FacultyID
CourseID → SemesterID
CourseID → ClassroomID

Faculty FDs

FacultyID → FacultyName
FacultyID → DepartmentID

Semester

SemesterID → SemesterNumber
SemesterID → AcademicYear

Classroom

RoomID → Building
RoomID → Capacity

Enrollment

(StudentID, CourseID) → Grade

(StudentID, CourseID) → Attendance

Step 2 — Candidate Keys

Relation	Candidate Key
Student	StudentID
Department	DepartmentID
Faculty	FacultyID
Course	CourseID
Classroom	RoomID
Semester	SemesterID
Enrollment	(StudentID, CourseID)

Convert to First Normal Form (1NF)

Rule: Every attribute must be atomic.

Original

StudentID	Phones
101	9876,9123

Convert

StudentPhone

StudentID	Phone
101	9876
101	9123

Original

StudentID	CourseIDs
101	CS301,EC204

Convert

Enrollment

StudentID	CourseID
101	CS301
101	EC204

Step 4 — Convert to Second Normal Form (2NF)

2NF removes Partial Dependencies.

Current Enrollment

|StudentID|CourseID|StudentName|CourseName|FacultyName|Grade|

Candidate Key

(StudentID, CourseID)

Functional Dependencies

StudentID → StudentName

CourseID → CourseName

CourseID → FacultyName

(StudentID, CourseID) → Grade

Problems:

StudentName depends only on StudentID.

CourseName depends only on CourseID.

FacultyName depends only on CourseID.

Partial Dependencies exist.

Split into

Student
| StudentID | StudentName | DepartmentID |

Course
| CourseID | CourseName | FacultyID |

Enrollment
| StudentID | CourseID | Grade |

Now every non-prime attribute depends on the whole key.
2NF achieved.

Convert to Third Normal Form (3NF)

Current Student

| StudentID | StudentName | DepartmentID | DepartmentName | HOD |

Functional Dependencies

StudentID $\rightarrow$ DepartmentID

DepartmentID $\rightarrow$ DepartmentName

DepartmentID $\rightarrow$ HOD

DepartmentName and HOD are transitively dependent.

Split

Student
| StudentID | StudentName | DepartmentID |

Department
| DepartmentID | DepartmentName | HOD |

Course table

Current

| CourseID | CourseName | FacultyName | SemesterNumber | Room |

FDs

CourseID $\rightarrow$ FacultyID

FacultyID $\rightarrow$ FacultyName

CourseID $\rightarrow$ SemesterID

SemesterID $\rightarrow$ SemesterNumber

CourseID $\rightarrow$ RoomID

RoomID $\rightarrow$ Building

Split further

Faculty

| FacultyID | FacultyName | DepartmentID |

Semester

| SemesterID | SemesterNumber | AcademicYear |

Classroom

| RoomID | Building | Capacity |

Course

| CourseID | CourseName | FacultyID | SemesterID | RoomID |

3NF achieved.

Check BCNF

Now inspect every FD.

Example

Faculty

FacultyID $\rightarrow$ FacultyName

FacultyID $\rightarrow$ DepartmentID

FacultyID is Candidate Key.

BCNF satisfied.

Department

DepartmentID $\rightarrow$ HOD

DepartmentID is Candidate Key.

BCNF satisfied.

Course

CourseID $\rightarrow$ FacultyID

CourseID is Candidate Key.

BCNF satisfied.

Enrollment

(StudentID, CourseID) $\rightarrow$ Grade

Composite key determines Grade.

BCNF satisfied.

No BCNF decomposition required.

Check Fourth Normal Form (4NF)

Suppose the college also stores:

- Student
- Skills
- Languages

Example

Student	Skill	Language
Aadya	C++	English
Aadya	C++	Hindi
Aadya	MATLAB	English
Aadya	MATLAB	Hindi

Independent facts.

Normalize

StudentSkill
|StudentID|Skill|

StudentLanguage
|StudentID|Language|

Now 4NF satisfied.

If such independent multi-valued facts are not stored, no 4NF decomposition is needed.

Check Fifth Normal Form (5NF)

Suppose internships require:

- Student
- Company
- Project
- Mentor

If these relationships are independent and reconstructable through joins, we may decompose further.

In a typical college management system, such Join Dependencies rarely exist.

Therefore:

No 5NF decomposition required.

Final Database Schema

Department

DepartmentID (PK)

DepartmentName

HOD

Student

StudentID (PK)

StudentName

DepartmentID (FK)

StudentPhone

StudentID (FK)

Phone

PK(StudentID, Phone)

Faculty

FacultyID (PK)

FacultyName

DepartmentID (FK)

Semester

SemesterID (PK)

SemesterNumber

AcademicYear

Classroom

RoomID (PK)

Building

Capacity

Course

CourseID (PK)
CourseName
Credits
FacultyID (FK)
SemesterID (FK)
RoomID (FK)

Enrollment

StudentID (FK)
CourseID (FK)
Grade
Attendance
PK(StudentID, CourseID)

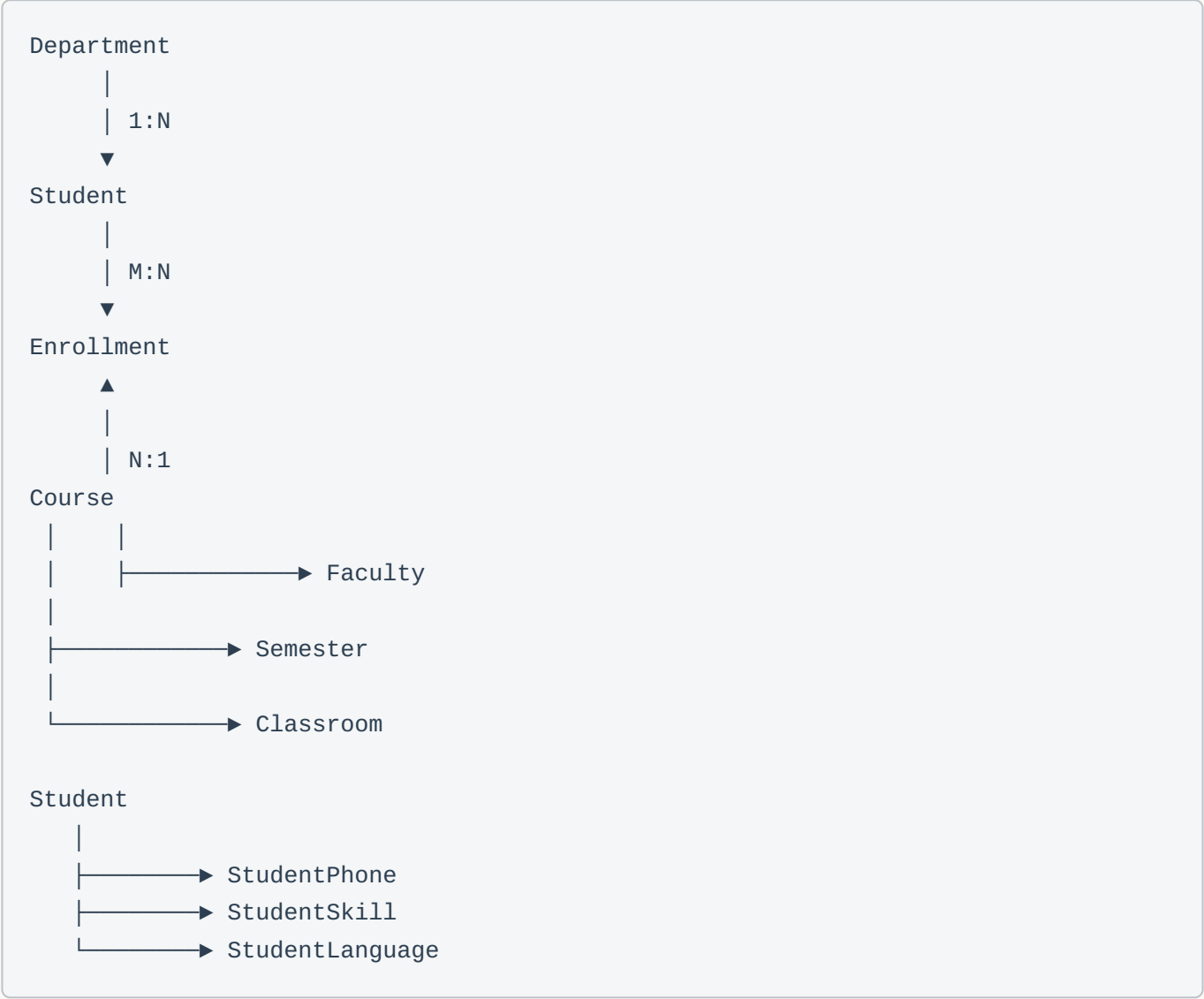
StudentSkill

StudentID (FK)
Skill
PK(StudentID, Skill)

StudentLanguage

StudentID (FK)
Language
PK(StudentID, Language)

Final ER-style Relationship Overview



SQL Table Creation

```
CREATE TABLE Department (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(100),  
    HOD VARCHAR(100)  
);  
  
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(100),  
    DepartmentID INT,  
    FOREIGN KEY (DepartmentID)  
        REFERENCES Department(DepartmentID)  
);  
  
CREATE TABLE StudentPhone (  
    StudentID INT,  
    Phone VARCHAR(15),  
    PRIMARY KEY(StudentID, Phone),  
    FOREIGN KEY(StudentID)  
        REFERENCES Student(StudentID)  
);  
  
CREATE TABLE Faculty (  
    FacultyID INT PRIMARY KEY,  
    FacultyName VARCHAR(100),  
    DepartmentID INT,  
    FOREIGN KEY(DepartmentID)  
        REFERENCES Department(DepartmentID)  
);  
  
CREATE TABLE Semester (  
    SemesterID INT PRIMARY KEY,  
    SemesterNumber INT,  
    AcademicYear VARCHAR(20)  
);  
  
CREATE TABLE Classroom (  
    RoomID INT PRIMARY KEY,  
    Building VARCHAR(50),  
    Capacity INT  
);  
  
CREATE TABLE Course (  
    CourseID VARCHAR(20) PRIMARY KEY,  
    CourseName VARCHAR(100),  
    Credits INT,
```

```
FacultyID INT,  
SemesterID INT,  
RoomID INT,  
FOREIGN KEY(FacultyID) REFERENCES Faculty(FacultyID),  
FOREIGN KEY(SemesterID) REFERENCES Semester(SemesterID),  
FOREIGN KEY(RoomID) REFERENCES Classroom(RoomID)  
);  
  
CREATE TABLE Enrollment (  
    StudentID INT,  
    CourseID VARCHAR(20),  
    Grade CHAR(2),  
    Attendance DECIMAL(5,2),  
    PRIMARY KEY(StudentID, CourseID),  
    FOREIGN KEY(StudentID) REFERENCES Student(StudentID),  
    FOREIGN KEY(CourseID) REFERENCES Course(CourseID)  
);  
  
CREATE TABLE StudentSkill (  
    StudentID INT,  
    Skill VARCHAR(50),  
    PRIMARY KEY(StudentID, Skill),  
    FOREIGN KEY(StudentID) REFERENCES Student(StudentID)  
);  
  
CREATE TABLE StudentLanguage (  
    StudentID INT,  
    Language VARCHAR(50),  
    PRIMARY KEY(StudentID, Language),  
    FOREIGN KEY(StudentID) REFERENCES Student(StudentID)  
);
```


Explanation of Every Decomposition Step

Step	Why it was Needed	Result
UNF → 1NF	Removed repeating groups (phones, courses) and made every attribute atomic.	Created StudentPhone and Enrollment.
1NF → 2NF	Removed Partial Dependencies caused by the composite key (StudentID, CourseID).	Split student details into Student, course details into Course, leaving only enrollment-specific attributes in Enrollment.
2NF → 3NF	Removed Transitive Dependencies such as DepartmentID → DepartmentName, FacultyID → FacultyName, and SemesterID → SemesterNumber.	Created separate Department, Faculty, Semester, and Classroom tables.
3NF → BCNF	Checked whether every determinant was a Candidate Key.	No violations found; schema already satisfied BCNF.
BCNF → 4NF	Checked for independent Multivalued Dependencies like student skills and languages.	Added StudentSkill and StudentLanguage only because those independent facts exist.
4NF → 5NF	Checked for Join Dependencies requiring further decomposition.	No such dependencies in the core college system, so no changes were needed.

Final Normalization Status

Normal Form	Status	Reason
1NF	✓	All attributes are atomic; no repeating groups.
2NF	✓	No Partial Dependencies remain.
3NF	✓	No Transitive Dependencies remain.
BCNF	✓	Every determinant is a Candidate Key.
4NF	✓	Independent multivalued facts (skills, languages) are separated.
5NF	✓	No non-trivial Join Dependencies exist in the design.